

Project Report - 2

Skbalaga (Sandeep Balaga) & smittal6(Swastik Mittal)

Point to Multipoint File Transfer Protocol

Topology

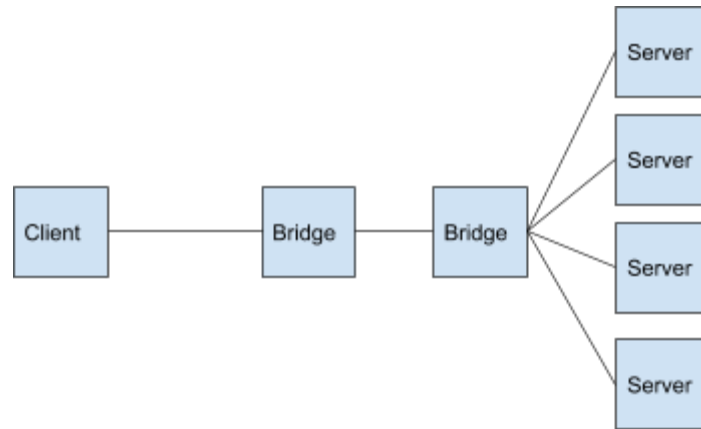


Fig 1

To run our experiments we implemented the above topology using containers where we use two containers as bridges (as in Fig 1), one as a client and other containers as the server. The two bridges create a multi hop scenario in our setup in order to provide close to realistic network environment.

Description mentions to form packet header while the file is being read till maximum segment size. We read 100 bytes chunk constantly for all the three tasks and for every 100 bytes add it to the sum value which is later reflected in the checksum. We also have a conditional statement after every 100 bytes read from file to check if total read is equal to mss or not.

Task1

Constraints: MSS = 500 bytes; $p = 0.05$; varying n from 1-5

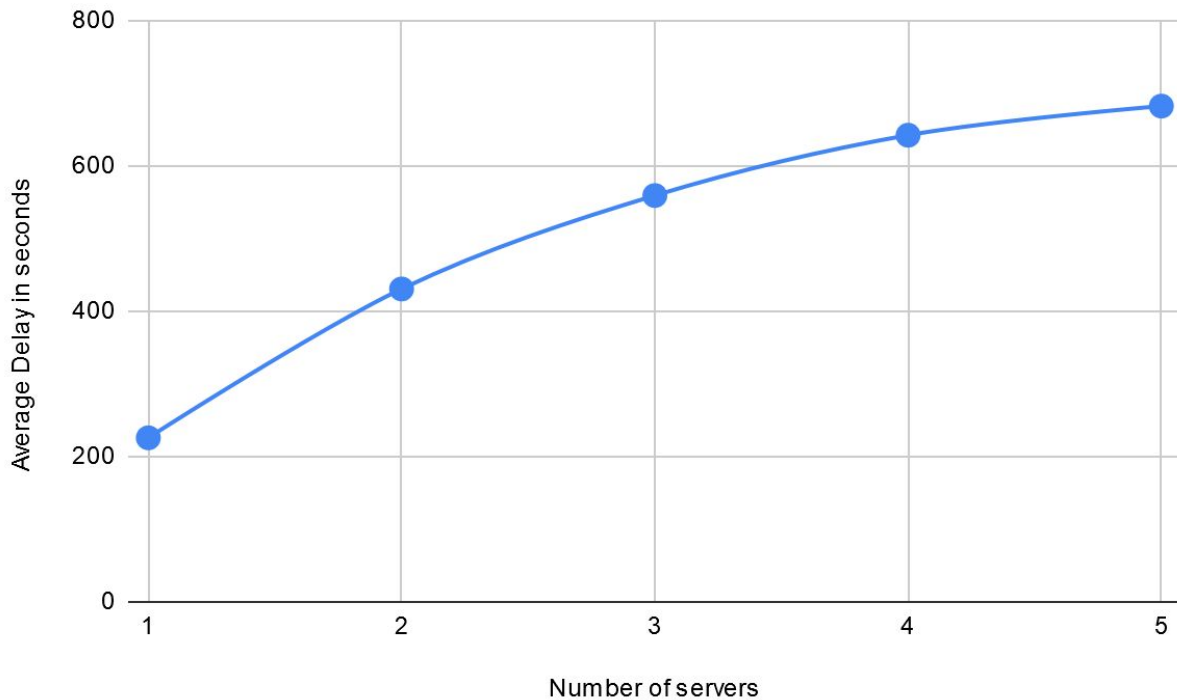


Fig 2

From Fig 2, we can observe that as we increase the value of n the average time to complete a file transfer increases. This is because the way we implement our reliable file transfer protocol stop and wait. So, once the client sends a packet of mss size to all the servers it stops till it receives ACK from all the servers. Now as servers maintain an individual loss probability of 0.05 there are situations where some of the servers may not receive any packet hence would not send any ACK. This would cause unnecessary delay to other servers who have already send their ACK's successfully increasing the overall time to transfer files. However, this value could be subjective given that there is a random probability of 0.05 of packet getting dropped. There may be times where packet on single server getting dropped was more frequent than when running multiple servers (irregular drops on the servers) which would give us some outlier values where transfer time might be less for more servers on the network. But as the given probability of loss is too low and we also average out the transfer time over 5 files it is highly unlikely to have such a situation.

Task 2

Constraints: $n=3$; $p= 0.05$; varying MSS from 100 - 1000

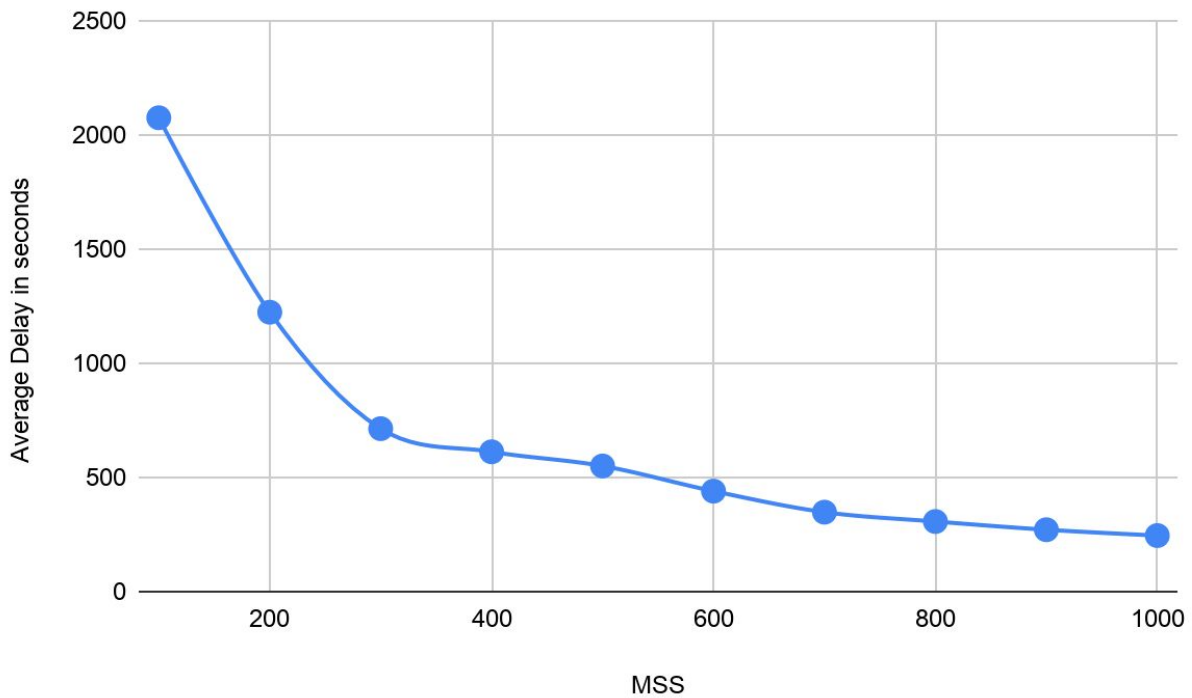
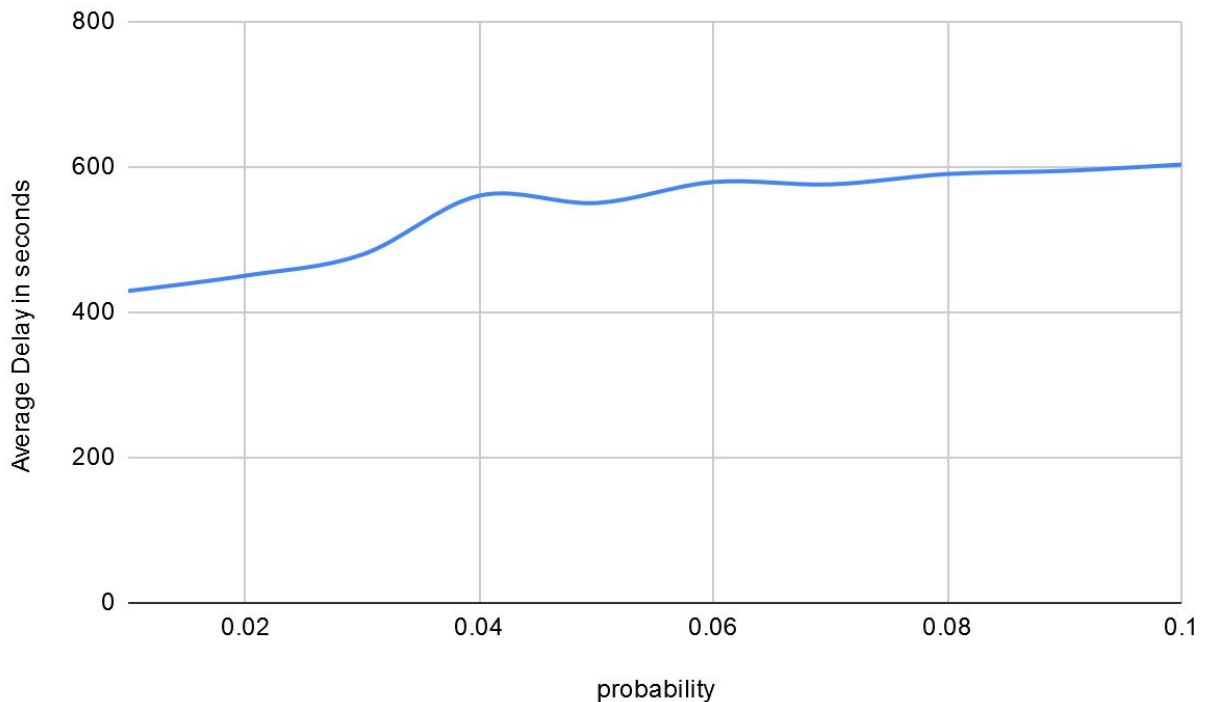


Fig 3

As expected, as we increase the MSS size the number of packets on the network decreases as larger chunks are sent to the server because of which the overall average file transmission time decreases. Larger chunks being lost would mean large data being retransmitted again but given the low loss probability this case does not affect the performance. Instead with fewer packets on the network less time is consumed to handle the acknowledgement and also less number of retransmissions expected hence as MSS increases average transmission time of the file decreases. (observed from fig3)

Task 3



(in this task we decrease the timeout value to 0.05sec which was initially set to 0.2 sec to get quicker results)

Constraints $n=3$; $MSS= 500$; varying p from 0.01-0.10

We get a very irregular graph for this task. We expected an increase in the average file transmission time as we increase the loss probability as more amount of redundant packets would be sent causing a delay in sending new chunks, making the whole process slower. However we can see a slight increase in the average transmission time but we get an irregular graph probably because we tried to tamper the timeout to a very low value because of which retransmission happens a lot even if losses are low.
